

Smart Traffic Light Controller using Deep Reinforcement Learning

Abhishek A
Dept. of Computer Science Engineering
PES University
Bengaluru, India
abhishek.adinarayan@gmail.com

Prathvik Nayak
Dept. of Computer Science Engineering
PES University
Bengaluru, India
prtvk13@gmail.com

Krishna P Hegde
Dept. of Computer Science Engineering
PES University
Bengaluru, India
krishnahegde02@gmail.com

A Lakshmi Prasad
Dept. of Computer Science Engineering
PES University
Bengaluru, India
avineniprasad199@gmail.com

Nagegowda K S
Dept. of Computer Science Engineering
PES University
Bengaluru, India
nagegowda@pes.edu

Abstract—Transportation has become a major priority for people nowadays. While technological advancements have made transportation easier, monitoring and controlling traffic as well as simulating it has become a significant challenge despite the significant research that has been carried out to tackle this problem. An emerging trend to solve this problem involves using deep reinforcement learning techniques which has shown significant progress and promising results in recent studies. There is an increasing demand for developing an intelligent traffic controlling agent which can dynamically adjust to real time traffic rather than just operating by hand-craft rules such as timers. We employ modern deep reinforcement learning methods such as Q-learning and deep neural networks with an agent-based traffic simulator SUMO ("Simulation of Urban MObility") which provides a synthetic yet realistic environment for simulating real-world like traffic and explore the outcomes of the actions that were performed on this environment. Additionally we also aim to analyze the performance of the agent by comparing it to traditional traffic signals.

Keywords—Reinforcement learning, traffic controlling agent, Q-Learning, deep neural network, SUMO

I. INTRODUCTION

We know pretty well for a fact that modern society depends heavily on transportation systems. Everyone desires to move from their origin to the destination in a smooth manner as possible, but due to the increase in population and the number of vehicles, traffic congestion, travel delay, accidents and unnecessary emissions have increased substantially and due to this there is a huge demand for improving the road infrastructure to ensure smooth flow of traffic. The fact that the world is changing and developing very fast makes it a difficult task to control and monitor the traffic, especially in large metropolitan areas that are growing around the world such as Tokyo, Bengaluru, Los Angeles, Mumbai, etc.

Majority of the traffic congestions occur due to lack of capacity on the roads i.e the number of vehicles that are traveling on the roads are exceeding their capacity. Apart from this, environmental issues, human errors, etc are other causes for traffic congestion. Solving this problem of traffic control and monitoring and, in general, reducing traffic congestion still remains a major challenge in spite of substantial research that has been carried out to tackle this problem.

Artificial Intelligence plays a key role in tackling this problem. We attempt to try and develop a smart traffic signal controller that is able to analyze and adjust dynamically to real-time traffic rather than just following traditional hand-craft rules such as timers.

Reinforcement learning could be an optimal technique to solve this problem of traffic congestion and traffic control. Reinforcement learning is one of the ML techniques which comprises an agent, state representation, actions set, Q-function and a reward function. The agent perceives the environment through its sensors and depending on that state of the environment, it performs a certain action. If the action taken was a good one, it gets a positive reward else it gets a negative reward. The agent has a goal which it achieves by maximizing the rewards.

In this paper, we have referred to the ideas proposed by Andrea Vidali et al. [1] to use a reinforcement learning technique called deep Q learning which is a mix of deep neural networks and Q-Learning. The approach is to use discrete traffic state encoding (DTSE) for state space representation of traffic and use an epsilon greedy policy along with experience replay mechanism for training purposes. We have extended this approach using different models like Artificial neural networks (ANN) and Convolutional neural networks (CNN) for different types of networks namely 3-way, 4-way, 5-way intersections and complex combinations of these intersections.

We will be using the SUMO traffic simulator for simulating the traffic. "Simulation of Urban MObility" (SUMO) is an open source, highly portable, microscopic and continuous traffic simulation package designed to handle large networks. It allows for intermodal simulation including pedestrians and comes with a large set of tools for scenario creation. We will be using the TraCI API for interacting with SUMO using python. TraCI follows a client server architecture based on TCP, where the controller (external script) acts as a client and SUMO software acts as a server. The "controller" is an external python script which gets the simulation state information from the server (SUMO) and then responds by sending instructions [5].

II. LITERATURE SURVEY

The research that has been conducted on traffic signal control using reinforcement learning has been significant.

The limitations of the earlier attempts include absence of sufficient compute power and simple simulations [6][7]. Starting from the early 2000's, the constant advancement in these areas has led to the creation of various simulation tools which are more realistic and sophisticated. Traffic simulators have become a popular tool among traffic researchers since they can replicate traffic behavior from the real-world. The research held has varied in the type of reinforcement learning, definition of state space, definition of action space, definition of reward, the simulator used, the geometry of the traffic network and the model for generation of vehicle. The number of queued vehicles [7][8][9] and traffic flow [10] are the most popular attributes of traffic that were defined for the state space in earlier research attempts. All available signal phases [10] or restricted to green phases only [8][9], have been the defined action space. Change in delay [10] and change in queued vehicles [8][9] are among the frequently used reward definitions.

Andrea Vidali et al. worked on simulation of traffic, as well as traffic monitoring and control using reinforcement learning approach with the help of a microscopic agent-based simulator [1]. Here they have used the DTSE [2] method for encoding the state information and have used a four way intersection as their environment. They have generated traffic according to Weibull distribution and have simulated different levels of traffic as different traffic scenarios. They have made use of two different reward functions: i) literature reward function - which uses *total waiting time* as metric, and ii) alternate reward function - which uses *accumulated total waiting time* as metric. The learning technique used here was Deep Q learning using an artificial neural network and the training was done using an epsilon greedy policy with experience replay mechanism. The performance was assessed in two parts: analyzing the reward trend and then comparing it with regular traffic signals. It was found that the alternative reward function gave better results than the literature reward function and it was seen that the agent performed better in low to medium traffic situations which led to think that the agents need to be separately developed to different traffic situations so that it can select the most appropriate configuration.

The Pennsylvania State University scholars have conducted research on intelligent traffic light control using reinforcement learning [3]. The experiment was conducted on a simulation platform SUMO which provides flexible APIs for road network design, traffic volume simulation and traffic light control. The evaluation metrics were reward, queue length, delay and duration. They compared methods of fixed-time control, self organizing Traffic light control, and deep reinforcement learning for traffic light control. In addition to the baseline methods, they considered several other variations of the model by adding memory palace and phase gates to the base model. The datasets that were used for conducting the experiments were synthetic data and real world data. They conducted in-depth analysis on the performance of all the proposed models with different constraints on the dataset.

In the research carried out by Wade Genders, they developed an agent for traffic signal control using a Deep Q-network (DQTSCA) [2] and a deep convolutional neural network (CNN) was used to model the action-value function and reinforcement learning technique was used to train it in an open source traffic simulator software called SUMO,

using an intersection which is isolated [2]. They have proposed a new definition for the state space called discrete traffic state encoding (DTSE) [2] which contains more relevant information as it is an improved representation of traffic. The CNN will be able to perceive information about the traffic that is more relevant with the help of DTSE. To make a comparison against their proposed DQTSCA, they also developed a shallow traffic signal control agent (STSCA) [2]. From the results obtained it was clear that the STSCA was outperformed by the DQTSCA, mainly because of the utilization of the DTSE and its deep architecture [2].

In the research carried out by Arel C. Liu¹ T. Urbanik, A.G.Kohls [4], it has been observed that multi-crossing point traffic organization, clogging in a solitary path doesn't just affect upstream traffic, yet in addition different convergences. Hence, a proficient strategy that can improve the reward and could handle traffic issues was desperately needed. In this research [4], a multi-agent setup is being used, whereby Reinforcement Learning is used as a method for handling the various intersections in the traffic system. A RL issue is characterized once states, actions, rewards are plainly characterized. Keeping that in mind, these fundamental builds are depicted with regards to the problem. At every simulation time step, local traffic statistics influence the local state of an intersection. An eight-dimensional feature vector is used for indicating each state with each element indicating the relative flow of traffic at any one lane. Relative traffic flow can be measured by dividing the total delay of vehicles with the average delay. However, a central junction agent can have access to all its neighboring junction states, thus such extra data permits it to anticipate upstream traffic streams, subsequently increasing the performance simulation results. One time step is referred to as one time unit in the considered environment. One discrete second is referred to as one time step. Once every 20 time units an action is taken, also only one vehicle is permitted to cross junctions at a given interval. The approach mentioned was created while seeing the traffic issue artificial intelligence task, indicating a shift from traditional traffic scheduling issue formulations. The basic goal here was to increase expected reward points which directly influences in reducing traffic issues.

However, in some of these researches the solution is limited to a single intersection (generally 4 way) using a single model. Also these researches do not consider priorities of emergency and authority vehicles in the simulations. In our project we aim to expand these solutions to different intersections and also to multiple intersections in a complex network using different models such as ANN and CNN. We will be analyzing the performance of these agents compared to traditional traffic signals and also study it's reward trends and its effects on undertraining in a complex network. We will also be studying the performance of these models in different intersections and also determine which type of model performed better than the other.

III. PROPOSED SOLUTION

Description of the Reinforcement Learning approach used in our project is as follows:

State representation: We followed the Discrete Traffic State Encoding (DTSE) to represent our state. In this encoding each incoming lane is divided into cells that can identify the presence or absence of a vehicle inside them.

Simulated Traffic



Position Matrix

0	0	0	0	0	0	0	1	1	1
0	0	0	0	0	0	0	1	0	0
0	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	1	1

Velocity Matrix

0	0	0	0	0	0	0	0.2	0	0
0	0	0	0	0	0	0	0.4	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0.8	0	0	0	0	0	0.2	0

Fig. 1. Discrete Traffic State Encoding (DTSE)

Action set: The possible actions that can be taken by the agent are contained in the action set. The number of actions in the action set is the same as the number of incoming lanes in an intersection. For a 4-way intersection there are 4 possible actions and each action is a phase of the traffic signal that is North Advance, East Advance, South Advance and West Advance. Similarity 3-way will have 3 actions in its action set and 5-way will have 5 actions in its action set.

Reward: Reward is something which the agent receives as a consequence of its chosen action. A good action results in a positive reward while a bad action results in a negative reward. We have used change in cumulative waiting time between actions as the metric to calculate the reward.

Learning mechanism: The learning mechanism used is Deep Q-Learning. It is a form of model free learning, which assigns a Q-value also known as quality value to an action that was performed on the state of the environment.

In literature, the Q-value is obtained from the Bellman's equation defined as in equation (1).

$$Q(s_t, a_t) = Q(s_t, a_t) + \alpha (\text{reward} + \gamma \cdot \max_A Q(s_{t+1}, a_t) - Q(s_t, a_t)) \quad (1)$$

Here $Q(s_t, a_t)$ is the Q-value for an action a_t taken on state s_t . We are updating the current Q-value with a quantity discounted by the learning rate (α). The immediate future's Q-value is represented by $Q(s_{t+1}, a_t)$ and $\max_A$ tells that among all the possible actions a_t in state s_{t+1} , the one with maximum value is selected. The term γ is the discount factor having a value between 0 and 1 to, this gives more importance to the immediate reward than the future reward.

In Q learning performs well in small state-spaces but its performance falls off when working with complex sophisticated environments. In small state spaces we just need to update a few states iteratively in the q-table, but when we are working in complex and large environments, the state-space is very large and it's computationally inefficient to update each state-action pair in the state space iteratively. Now instead of using value iteration to directly compute the Q values and find the optimal Q function, instead we use a function approximator to estimate the optimal Q function, hence neural networks come into picture. The output Q values from the neural network (one for each action) is compared to the q values obtained from the Bellman's equation to compute loss. This loss is minimized using back propagation and tweaking the weights using stochastic gradient descent using Adam optimizer.

Training Process: We have used the experienced relay technique to enhance performance and efficiency. This technique consists of submitting all the knowledge needed by the agent for the training process as a group or collection of random samples also called as a batch rather than instantly submitting the knowledge gathered by the agent during the phase of simulation. All the samples collected during the training phase are stored in the memory and a group of samples are retrieved from the memory as a batch. Training includes iterative learning of a Q-value function with the help of knowledge available within the batch of samples retrieved from the memory.

In this paper, we have used a slightly modified version of equation (1) which is presented in the form of equation (2).

$$Q(s_t, a_t) = \text{reward} + \gamma \cdot \max_A Q'(s_{t+1}, a_{t+1}) \quad (2)$$

The action-selection policy we used is the epsilon-greedy policy. This is the policy that decides whether an agent performs an exploratory action where the agent tries something new, or performs an exploitative action where the agent exploits what it already knows to achieve favorable rewards.

Further during the testing phase, we tested the trained model with random traffic simulations to see its performance. Here we also configured our emergency vehicles like ambulances, fire trucks, police vehicles, etc with blue light devices in SUMO to ignore traffic signals so that they don't have to wait unnecessarily. By doing this any vehicle can drive with special rights and change lanes without regard for strategic considerations. All the other vehicles ahead of the emergency vehicles are forced to form a rescue lane to let the emergency vehicle pass through.

IV. RESULTS AND DISCUSSION

On visual inspection it was clearly seen that the trained model performed much better than the normal simulation, as the trained model had learnt an optimal policy function to maximize the rewards and choose an optimal action.

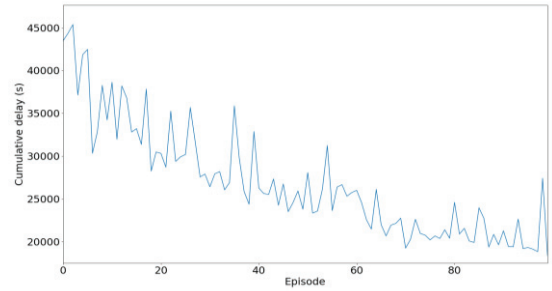


Fig. 2. Cumulative delay(s) vs. Episode graph

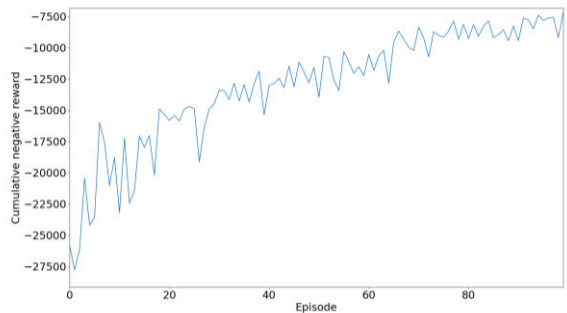


Fig. 3. Cumulative negative reward vs. Episode graph

Our agents were able to reduce the cumulative delay of vehicles and maximize the rewards during the training process as we can see in the below graphs (Fig. 2 & Fig. 3).

In comparison with traditional traffic signals it was seen that the trained agent was able to reduce queuing in traffic signals significantly as seen in Fig. 4.

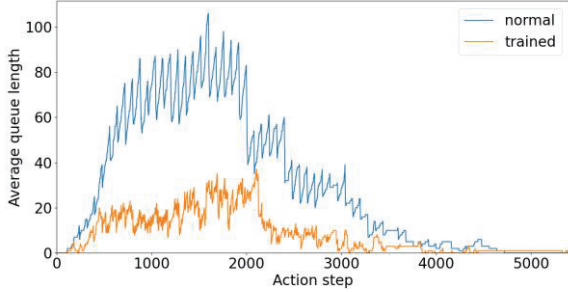


Fig. 4. Average queue length vs. Action step graph (normal vs. trained)

The reason for this reduction in queue lengths as seen in Fig. 4 is the duration of the traffic light phases. Here we have chosen a normal scenario wherein the duration of green and yellow phases is 30 seconds. During training, apart from the agent choosing an appropriate action, we are making sure that the green light duration for a particular lane does not exceed 10 seconds and the yellow light duration does not exceed 4 seconds. By doing this it allows the agent to take quicker actions depending on the surroundings. In case the action chosen at actionstep t is the same as the action taken at actionstep $t-1$, then the green light persists and there is no yellow phase, hence there is only 10 seconds (10 simulation steps between the two actions). Else if the consecutive actions are not the same then there is a 4 second yellow phase between the two actions making it a total of 14 seconds (14 simulation steps) between the two actions.

On analyzing the performance of the agents, it was seen that the agents trained using the CNN performed much better than the agents trained using the artificial neural network as seen in Fig. 5, Fig. 6, Fig. 7 & Fig. 8.

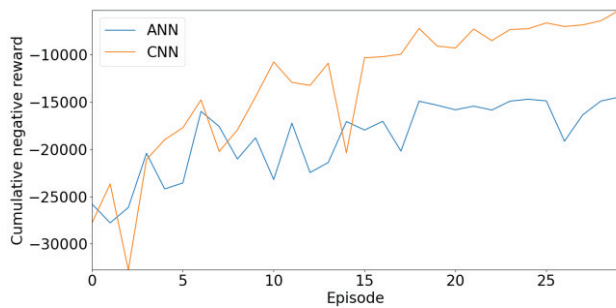


Fig. 5. Cumulative negative reward vs. Episode (ANN and CNN)

On analysis of the reward trend we can see that the CNN is able to maximize the rewards much quicker compared to ANN with lesser training (Fig. 5).

When cumulative delay is plotted for both ANN and CNN, we can clearly see that CNN has lesser cumulative delay when compared to ANN (Fig. 6).

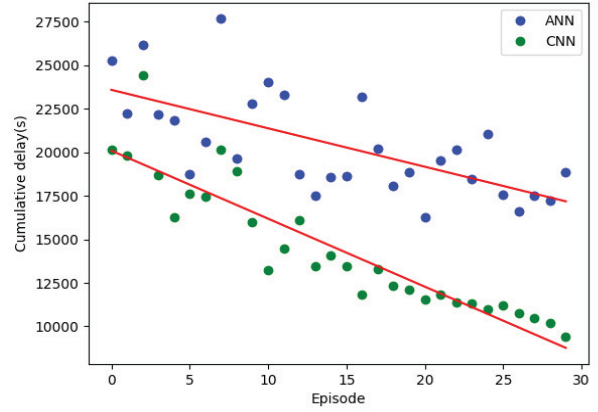


Fig. 6. Cumulative delay(s) vs. Episode (ANN and CNN)

As we can see from Fig. 7 and Fig. 8 the CNN agent is able to increase the negative reward and reduce the average queue length much quicker than the ANN as the number of episodes increases.

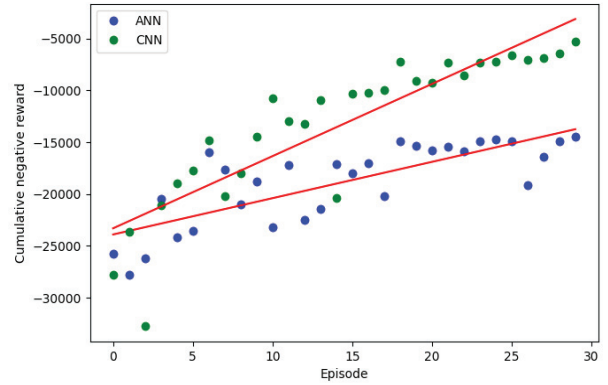


Fig. 7. Cumulative negative reward vs. Episode (ANN and CNN)

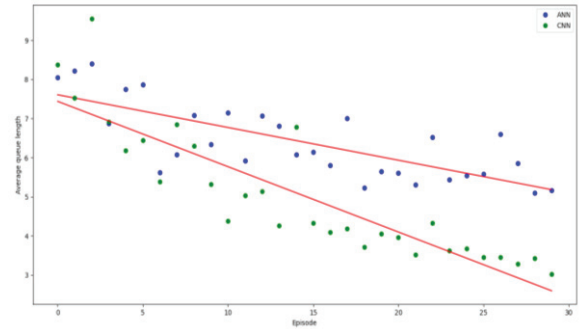


Fig. 8. Average queue length vs. Episode (ANN and CNN)

The reason why CNN performs much better compared to ANN is because of its ability to extract important features using DTSE. This allows CNN to develop high-level state representations. Moreover the CNN takes multiple data inputs, in our case the position and velocity matrix compared to the position matrix alone in the ANN. This allows CNNs to achieve better results with higher accuracy. Although CNN takes longer to train, it gives efficient results much quicker.

It was also seen that the models performed better in smaller intersections than the bigger ones (Fig. 9 & Fig. 10).

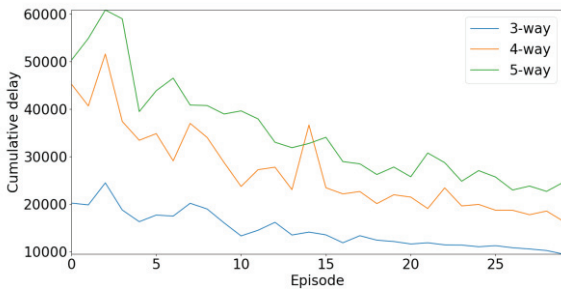


Fig. 9. Cumulative delay(s) vs. Episode

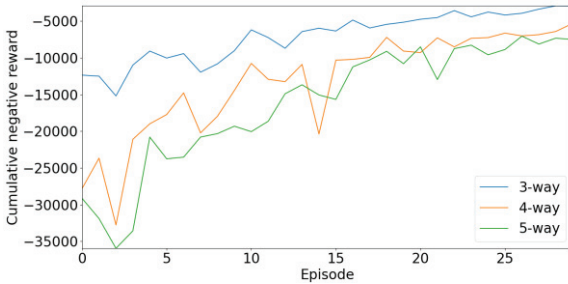


Fig. 10. Cumulative negative reward vs. Episode (3-way, 4-way and 5-way)

This is mainly due to the fact that in smaller intersections, the state space representation is smaller compared to larger intersections and the model needs to choose from a smaller action set making it easier and quicker to train.

In complex intersection networks, when one model is undertrained, we found that it performs worse when compared to the fully trained models. The queue_length keeps on increasing for the partially trained model because of the congestion in the undertrained network. This can be seen from the plot below (Fig. 11).

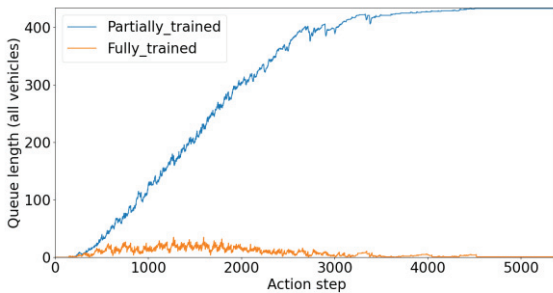


Fig. 11. Queue length vs. Action Step (Fully trained vs partially trained)

The above figure (Fig. 11) shows that while training for complex networks, each model has to be trained fully to achieve better results. Even if one of them is undertrained, the whole network will perform worse, since each individual network is interconnected and queue_length in one network impacts the whole network.

V. CONCLUSION

The work carried out presented a RL approach to monitor and control traffic lights in a much more effective manner wherein they are able to adjust dynamically to real time traffic and take appropriate actions. This work was carried

out on a traffic simulator which provides a realistic environment to carry out complex simulations. The agents were trained using the DQN mechanism and its performance was analyzed using various parameters. It was important to properly understand the application context and apply them to machine learning models to achieve proper results.

This work has shown that proper use of Machine learning approaches can provide effective and excellent results. Reinforcement learning, despite its limited application in the real world, is still one of the hot topics in the field of Artificial Intelligence and could potentially have a great impact in the future.

There are however many things that can be improved in this work. Future works could include improving the results achieved and expanding it to a much larger network. The training of the agents could be improved by using different techniques such as target network, clipping rewards and skipping frames. We can also try using different reward and Q functions to try and improve the model further. Proper analysis is important to understand the advantages and possible disadvantages that it can bring on introduction to the real world.

ACKNOWLEDGMENT

The authors would like to thank all the authors of the various sources that are used in this project. The authors would also like to thank PES University for their support and encouragement throughout this project and acknowledge the support of friends and family.

REFERENCES

- [1] Andrea Vidali, Luca Crociani, Giuseppe Vizzari, Stefania Bandini, "A Deep Reinforcement Learning Approach to Adaptive Traffic Lights Management".
- [2] Wade Genders, Saiedeh Razavi, "Using a Deep Reinforcement Learning Agent for Traffic Signal Control".
- [3] Hua Wei, Guanjie Zheng, Huaxiu Yao, Zhenhui Li, "IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control".
- [4] Arel C. Liu, T. Urbanik, A.G. Kohls, "Reinforcement learning-based multi-agent system for network traffic signal control".
- [5] SUMO documentation: <https://sumo.dlr.de/docs/> Copyright © 2001-2021 German Aerospace Center (DLR) and others.
- [6] T. L. Thorpe and C. W. Anderson, "Traffic light control using sarsa with three state representations," Citeseer, Tech. Rep., 1996.
- [7] M. Wiering et al., "Multi-agent reinforcement learning for traffic light control," in ICML, 2000, pp. 1151-1158.
- [8] Y. K. Chin, N. Bolong, A. Kiring, S. S. Yang, and K. T. K. Teo, "Q-learning based traffic optimization in management of signal timing plan," International Journal of Simulation, Systems, Science & Technology, vol. 12, no. 3, pp. 29-35, 2011.
- [9] M. Abdoos, N. Mozayani, and A. L. Bazzan, "Holonc multi-agent system for traffic signals control," Engineering Applications of Artificial Intelligence, vol. 26, no. 5, pp. 1575-1587, 2013.
- [10] I. Arel, C. Liu, T. Urbanik, and A. Kohls, "Reinforcement learning-based multi-agent system for network traffic signal control," IET Intelligent Transport Systems, vol. 4, no. 2, pp. 128-135, 2010.